

Projektbericht

Webtechnologien 2

Eingereicht von:

Minh Hieu Le

Ivan Pushkarev

Vladyslav Hudym

Rui Yan

Technische Universität Dortmund

Fakultät für Informatik

Inhaltsverzeichnis

Zusammenfassung 2

1 Konzept..... 3

2 Style..... 4

3 Frontend 5

4 Backend 6

Fazit 7

Zusammenfassung

Das Ziel des Projekts ist die Anwendung der Konzepte, die in der Vorlesung vorgestellt wurden, in die Entwicklung einer Webapp.

Das Projekt enthält

- eine Webanwendung, die mittels der vorgestellten Techniken entwickelt (Persistenz-Schicht mit JPA, Backend mit Spring, Frontend mit Angular, Security-Mechanismus mit Apache Shiro)
- die Testierung des Produktes mittels von ALEX

Die Webanwendung sollte eine ToDo-App. Mit der Anwendung sollte man sich registrieren und einloggen/ ausloggen können. Als eine To-Do-Liste sollte es unbedingt möglich sein, Aufgaben hinzuzufügen, zu ändern oder zu löschen.

1 Konzept

Idee

Eine App zu entwickeln, die als eine vollständige Todo-Liste funktioniert. Die App muss dabei intuitiv und selbsterklärend sein, damit die Nutzer die App problemlos benutzen könnten. Unsere App muss die folgende Funktionalität haben: Todos erstellen, löschen, editieren, als erledigt markieren, Datum auswählen. Außerdem muss eine Nutzer-Anmeldung implementiert werden, damit die User sich einloggen können. Weil die Daten in einer digitalen Datenbank gespeichert werden, kann man die ganze App-Funktionalität mit verschiedenen Geräten benutzen.

Erwartungen

- minimalistisches und modernes Design
- Unterstützung verschiedener Endgeräte
- simple und intuitive Funktionalität
- sichere Datenübertragung
- 100% user-friendly

Planung

1. Funktionalität überlegen
2. Design-Ideen sammeln
3. Zusammenhänge vorstellen und skizzieren
4. Front end implementieren
5. Back end implementieren
6. Front end zum Back end verbinden
7. Polishing und Refinement 8. Testen

2 Style

Idee

Die App muss minimalistisch und modern aussehen. Laut unser Vorstellung sollte jedes einziges Todo-Item als ein Eintrag im Kalender und als ein Item in der separaten Liste aussehen. Alle Informationen des Todo-Items müssen dabei auf dem List-Item angezeigt werden. Deswegen müssten wir uns für ein Style-Pattern entscheiden, das sowohl gut aussieht, als auch die Seite nicht überlastet. Die Auswahl war sehr breit, weil es viele verschiedene Styles gibt, die man nutzen könnte. Wir haben uns trotzdem für Microsoft-Office-Style entschieden. Es sieht wirklich schön und minimalistisch aus, ist vielen Nutzern bekannt und lässt sich effektiv für unsere Zwecke einsetzen. Das List-Item enthält dabei alle wichtigen Infos, ist aber von verschiedenen Elementen nicht überlastet.

Implementierung

Für die Implementierung der Styles haben wir Angular Material benutzt. Es ist eine von Google entwickelte Komponenten-Bibliothek, die den Developers erlaubt, die fertigen Komponenten schnell und problemlos in das Projekt zu importieren. Diese Bibliothek bietet eine breite Auswahl an verschiedenen Komponenten, die schon gestyled und animiert sind. Angular Material erleichtert den Entwicklungsprozess und garantiert dass alle benutzten Komponenten korrekt funktionieren.

Probleme

Zuerst haben wir die Angular-Material Komponenten nicht in ein Modul, sondern direkt in die Komponente importiert. Dieser Ansatz funktionierte aber wir haben bemerkt, dass manche Angular-Material Komponenten mehrmals importiert wurden. Es war kein großes Problem und die App funktionierte wie geplant aber irgendwie hatten wir ein Gefühl, dass die Imports nicht übersichtlich waren. Deswegen haben wir uns entschieden, alle Angular-Material Komponenten in Module zu importieren. So könnten wir die App-Struktur verbessern, ohne Funktionalität zu verlieren.

3 Frontend

Setup

Zuerst haben wir eine neue Angular 13 App erstellt. Dann wurden alle Default Einträge gelöscht, damit man das Projekt „from scratch“ bearbeiten kann. Danach haben wir uns überlegt, wie man die Dateien in dem Projekt strukturieren möchte, damit alles übersichtlich bleibt. Wir haben uns also Gedanken gemacht, welche Komponenten wir in dem Projekt haben wollen und wie diese Komponenten miteinander verbunden sind.

Komponenten

Für die App haben wir viele verschiedene Komponenten erstellt, die miteinander verbunden sind. Unser Ansatz sieht folgenderweise aus: wir haben mehrere Hauptkomponenten, die als Parent-Elemente für die anderen Komponenten gelten. Die wichtigsten Komponenten sind: LoginComponent, SignUpComponent, HomeComponent, PublicComponent, Admin-Component. Die Namen der Komponenten bedeuten in diesem Fall ihre Rollen. Jede Hauptkomponente bestimmt eine spezifische Route. Das heißt, dass LoginComponent für die Login-Seite zuständig ist usw. Alle Routes sind in der Datei `app-routing.module.ts` angegeben. Es gibt auch Komponente für „Add Task“-Form, Task-List-Item usw.

Entwicklungsprozess

Zuerst haben wir uns überlegt, wie die Routing-Modelle aussehen würde und welche Seiten wir in der App haben möchten. So könnten wir also von Anfang an wissen, wie wir die App strukturieren werden, damit der Entwicklungsprozess möglichst übersichtlich und klar ist. Als die wichtigsten Komponenten-Zusammenhänge implementiert wurden, haben wir angefangen, alle anderen Unterkomponenten zu erstellen. Dann haben wir Angular-Material importiert. Die importierten Komponenten haben wir erfolgreich in dem Projekt benutzt, um die gewünschte Funktionalität und Aussehen zu implementieren.

Testen

Um zu testen, ob Frontend funktioniert, erstellten wir eine Service, die das Back end simuliert. Wir haben also alle get-, post-, remove-Methoden lokal ausgeführt, wobei localStorage die Datenbank war. Außerdem erstellten wir ein Interface, das als Datenvorlage für jedes einzelne Todo-Item war. Dann importierten wir in der Parent-Komponente die Service, um die Daten aus der lokalen Datenbank zu holen. Dann wurde mit `*ngFor` für jedes Todo-Item aus localStorage ein eigener Eintrag in dem Kalender erstellt. Alles funktionierte perfekt.

4 Backend

Setup

Wir haben zur Backendentwicklung das Java-Framework Spring verwendet. Wir haben im *start.spring.io* ein neues Springboot Web-Projekt angefangen. Wir haben zuerst *Maven Project* abgehakt und natürlich Java als die Sprache ausgewählt. Dann haben wir die Version von Springboot bestimmt. Unter *Project Metadata* haben wir den Projektsnamen eingegeben. Anschließend haben wir die Java-Version, die bereits markiert, genommen. *Spring Web* haben wir mit *Add Dependencies* hinzugefügt. Nachdem alles schon wie oben beschrieben definiert war, wurde abschließend ein Template für unser Spring Projekt generiert.

Komponenten

Im *pom.xml* werden die Abhängigkeiten/ Dependencies definiert. Unter *src/main/resources* findet die Konfigurationsdatei *application.properties*, die zur Bestimmung der Properties der Spring Anwendung dient.

Entwicklungsprozess

Wir haben uns zuerst überlegt, welche Sachen und welche Klassen die Anwendung braucht. Die Klassen haben wir in 4 Foldern *controller*, *dao*, *entity* und *service* eingeteilt. Damit es für alle Gruppenmitglieder einfacher ist, wenn irgendwas zu verändernd sind.

Für Datenbank haben wir SQL verwendet, um die Daten zu speichern.

Testen

Um zu testen, ob Backend gut funktioniert, haben wir alle Funktionen-Hinzufügen, Verändern und Löschen probiert. Auch die Registrierung und das Einloggen mit registrierten Daten haben wir getestet. Alles hat perfekt und genauso wie erwartet funktioniert.

Fazit

Wir haben eine funktionierende App entwickelt, die den Nutzer eine vernünftige Todo-Verwaltung ermöglicht. Alle in Konzept-Teil beschriebenen Erwartungen haben wir erfüllt. Während des Entwicklungsprozesses haben wir viele neue Kompetenzen erworben und viel Erfahrung gesammelt, die uns in der Zukunft beim Erstellen weiteren Projekte helfen wird.